TOUCH GRASS

Technical Requirements Document (TRD)

Project type: AI-powered real-world challenge game
Version: 1.0 — MVP
Date: October 9, 2026
Primary language: TypeScript
Package manager: pnpm
Deployment target: Render
Database: MongoDB Atlas
AI model: Gemma
AI orchestration: Mastra

---

1. Project Overview

1.1 What is Touch Grass?

Touch Grass is a real-world challenge game that encourages young people to spend time outdoors through personalized AI-generated missions, evidence-based rewards, progression, and competition with friends.

Players choose the type of experience they want, such as exploring, relaxing, observing, socializing, or being active. The AI Game Master generates a challenge tailored to their preferences, available time, difficulty, and previous challenges.

The player then goes outside, completes the mission without needing to keep looking at their phone, returns to the app, submits the required evidence, and earns experience points (XP).

Players can also create shared Spaces with friends, complete group challenges independently, and compete based on their results.

Core product principle: The phone is the scoreboard. The game happens outside.

1.2 Core gameplay loop

Choose experience → Generate challenge → Go outside → Complete mission → Submit evidence → Evaluate submission → Award XP → Update progression and leaderboard.

1.3 Product objectives

1. Make outdoor activities engaging rather than feeling like exercise or chores.
2. Generate challenges that feel personal and different across sessions.
3. Encourage players to put their phones away while completing challenges.

4. Reward interesting discoveries, creativity, and successful completion.
5. Support both solo play and friendly competition.
6. Use AI for meaningful challenge generation and evidence evaluation.
7. Provide persistent player profiles, XP, levels, history, and leaderboards.

1.4 What the product is not

The MVP must not become:

- A fitness-tracking application.
- A navigation or route-planning application.
- A generic AI chatbot.
- A photo-sharing social network.
- A complex multiplayer game requiring users to meet at a physical location.
- A system that requires organizers to hide physical clues, QR codes, or stickers.

---

2. Technology Stack

Use TypeScript for the frontend, backend, shared packages, game logic, and AI integration.

2.1 Technology decisions

Layer	Technology	Purpose
Frontend framework	Next.js App Router	Application UI and routing
UI library	React	Components and interactions
Language	TypeScript	Type safety across the codebase
Styling	Tailwind CSS	Responsive styling
UI components	shadcn/ui + Radix UI	Accessible interface components
Icons	Lucide React	Icons
Forms	React Hook Form	Form state and submission
Validation	Zod	Runtime validation and shared schemas
API client state	TanStack Query	Server-state fetching and caching
Backend runtime	Node.js	JavaScript/TypeScript server runtime
Backend framework	Express.js	REST API
AI orchestration	Mastra	AI workflows and model orchestration
AI model	Gemma	Challenge generation and evidence evaluation

Layer	Technology	Purpose
AI connection	Provider adapter using an OpenAI-compatible API or official provider SDK	Model inference
Database	MongoDB Atlas	Persistent application data
Database driver	MongoDB Node.js driver	Database access
Authentication	JWT + bcryptjs	User sessions and password hashing
API security	Helmet, CORS, express-rate-limit	Security headers, origin restrictions, request limiting
Image storage	Cloudinary	Evidence photo storage
Monorepo	pnpm workspaces	Manage applications and packages
Monorepo task runner	Turborepo	Build and development tasks
Testing	Vitest	Unit and integration tests
API testing	Supertest	Express endpoint tests
End-to-end testing	Playwright	Browser-based flow tests
Deployment	Render	Host web application and API
Version control	Git + GitHub	Source control and repository hosting

2.2 Important AI hosting decision

Gemma is a model, not a standard npm package that automatically runs locally when installed.

The agent must implement a configurable model provider. It must verify which Gemma inference endpoint or hosting route is actually available and use a supported SDK or compatible API.

Use environment variables for:

- AI provider/base URL.
- Model identifier.
- API key, if required.
- Generation and evaluation settings.

Do not hard-code a specific inference vendor into business logic. Do not assume model inference is free merely because Gemma is open-weight.

If credentials are unavailable, the rest of the application must still start. Provide a clearly identified development mock provider for local UI and workflow testing. Never present mock AI output as a real Gemma result.

Gemma is the intended production model for the project. The mock is only a development fallback.

---

3. Repository Architecture

Use a pnpm monorepo. Do not scatter files across unrelated directories or put all application logic in one folder.

3.1 Required directory structure

Create the following structure:

```
touch-grass/
├── apps/
│   ├── web/
│   │   ├── public/
│   │   ├── src/
│   │   │   ├── app/
│   │   │   │   ├── (auth)/
│   │   │   │   ├── (game)/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   ├── challenge/
│   │   │   │   │   ├── complete/
│   │   │   │   │   ├── results/
│   │   │   │   │   ├── leaderboard/
│   │   │   │   │   ├── spaces/
│   │   │   │   │   └── profile/
│   │   │   │   ├── api/
│   │   │   │   ├── layout.tsx
│   │   │   │   └── globals.css
│   │   │   ├── components/
│   │   │   │   ├── ui/
│   │   │   │   ├── challenge/
│   │   │   │   ├── profile/
│   │   │   │   ├── leaderboard/
│   │   │   │   └── spaces/
│   │   │   ├── hooks/
│   │   │   ├── lib/
│   │   │   └── providers/
│   │   ├── next.config.ts
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   └── api/
│       ├── src/
│       │   ├── server.ts
│       │   ├── app.ts
│       │   ├── config/
│       │   │   └── env.ts
│       │   ├── routes/
│       │   ├── controllers/
│       │   ├── services/
│       │   ├── middleware/
│       │   ├── utils/
```

```
│       │   └── tests/
│       ├── package.json
│       └── tsconfig.json
│
├── packages/
│   ├── ai/
│   │   ├── src/
│   │   │   ├── agents/
│   │   │   ├── workflows/
│   │   │   ├── providers/
│   │   │   ├── prompts/
│   │   │   ├── schemas/
│   │   │   └── index.ts
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   ├── db/
│   │   ├── src/
│   │   │   ├── client.ts
│   │   │   ├── collections.ts
│   │   │   ├── indexes.ts
│   │   │   └── repositories/
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   ├── game/
│   │   ├── src/
│   │   │   ├── xp.ts
│   │   │   ├── levels.ts
│   │   │   ├── scoring.ts
│   │   │   ├── validation.ts
│   │   │   ├── leaderboard.ts
│   │   │   └── index.ts
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   └── shared/
│       ├── src/
│       │   ├── schemas/
│       │   ├── types/
│       │   ├── constants/
│       │   └── index.ts
│       ├── package.json
│       └── tsconfig.json
│
├── docs/
├── .env.example
├── .gitignore
├── package.json
├── pnpm-workspace.yaml
├── turbo.json
├── tsconfig.base.json
├── README.md
└── pnpm-lock.yaml
```

The agent may adjust individual filenames if necessary, but must preserve the separation of responsibilities.

3.2 Responsibilities

`apps/web` — frontend

Responsible for:

- Pages, navigation, layouts, and UI.
- Signup and login forms.
- Experience selection.
- Challenge instructions.
- Photo and text evidence submission.
- Results, XP, profiles, leaderboards, and Spaces.
- Calling the backend API.

The frontend must never access MongoDB directly or expose AI provider credentials.

`apps/api` — backend

Responsible for:

- REST endpoints.
- Authentication and authorization.
- Request validation.
- Challenge lifecycle management.
- Calling AI workflows.
- Evidence upload coordination.
- XP awarding and leaderboard updates.
- Space membership and group challenges.
- Error handling and security.

`packages/ai` — AI system

Responsible for:

- Mastra configuration.
- Gemma model provider.
- Challenge generation.
- Evidence evaluation.
- Prompt templates.
- Structured model outputs.
- AI response validation and retry handling.

`packages/db` — database

Responsible for:

- MongoDB connection and lifecycle.
- Collection access.
- Index creation.
- Data repositories.
- Database-specific queries.

`packages/game` — deterministic game engine

Responsible for:

- XP calculations.
- Bonus caps.
- Level progression.
- Challenge validity rules.
- Duplicate completion prevention.
- Leaderboard ranking rules.

This package must not depend on the AI model.

`packages/shared` — shared contracts

Responsible for:

- Shared TypeScript types.
- Zod request and response schemas.
- Challenge and evidence enums.
- API contracts.
- Shared constants.

Avoid circular dependencies between packages.

---

4. Monorepo Configuration

The coding agent must configure the workspace rather than simply creating empty directories.

4.1 Root `package.json`

Create root scripts for:

- `dev`: run the web app and API concurrently.
- `build`: build all applications and packages.
- `lint`: run linting.
- `typecheck`: type-check the workspace.
- `test`: run unit and integration tests.
- `test:e2e`: run Playwright tests.
- `format`: format supported source files.

Use Turborepo to coordinate tasks and pnpm to install dependencies.

4.2 `pnpm-workspace.yaml`

```
packages:
  - "apps/*"
  - "packages/*"
```

4.3 `turbo.json`

Configure tasks with correct dependency ordering:

- Build dependencies before dependent applications.
- Type-check all relevant packages.
- Run tests after required packages are built where applicable.
- Use persistent development tasks for the frontend and API.
- Avoid caching environment-specific secrets or runtime outputs.

4.4 TypeScript configuration

Create a shared `tsconfig.base.json` with strict TypeScript settings.

Requirements:

- Enable `strict`.
- Avoid `any` unless justified.
- Configure appropriate module resolution for Next.js and Node.js.
- Ensure packages can be imported through workspace package names.
- Use consistent aliases where appropriate.
- Ensure build output and TypeScript paths work in production, not only in the development server.

Use package-specific TypeScript configurations extending the root configuration.

---

5. Dependencies to Install

The coding agent must install compatible, currently supported versions of the dependencies below, using pnpm. It should check package compatibility before selecting versions rather than blindly pinning old versions.

5.1 Frontend dependencies

Install in `apps/web`:

```
next
react
react-dom
lucide-react
@tanstack/react-query
react-hook-form
@hookform/resolvers
```

```
zod
```

Use the current Next.js App Router and TypeScript template. Next.js already brings some React and TypeScript development dependencies; do not install conflicting duplicate versions.

Install Tailwind CSS and its required dependencies according to the selected Next.js template.

Initialize shadcn/ui in `apps/web` and install only the components actually used. Suitable components include:

- Button
- Card
- Dialog
- Dropdown Menu
- Form
- Input
- Label
- Select
- Tabs
- Avatar
- Badge
- Progress
- Skeleton
- Toast or the currently recommended notification component

Use Radix primitives through shadcn/ui where appropriate. Do not install a second full component library without a concrete reason.

Use TanStack Query for server state. Keep ordinary local UI state in React rather than introducing Redux unnecessarily.

5.2 Backend dependencies

Install in `apps/api`:

```
express
cors
dotenv
zod
helmet
express-rate-limit
jsonwebtoken
bcryptjs
mongodb
```

Development dependencies:

```
typescript
tsx
@types/node
```

```
@types/express
@types/cors
@types/jsonwebtoken
@types/bcryptjs
vitest
supertest
@types/supertest
```

Use versions compatible with the selected Express and TypeScript releases.

Responsibilities:

- `express`: REST API.
- `cors`: restrict browser access to approved frontend origins.
- `dotenv`: load local environment configuration.
- `zod`: validate request data and environment configuration.
- `helmet`: set security-related HTTP headers.
- `express-rate-limit`: restrict abusive requests.
- `jsonwebtoken`: issue and verify access tokens.
- `bcryptjs`: hash passwords securely.
- `mongodb`: connect to Atlas.
- `tsx`: execute TypeScript during development.
- `vitest` and `supertest`: test backend behavior.

Do not install Fastify. The backend framework is Express.js.

5.3 AI dependencies

Install in `packages/ai`:

```
@mastra/core
zod
```

Also install the specific Mastra model-provider integration required by the chosen Gemma inference endpoint, if applicable.

Before installing, the agent must consult the current Mastra documentation and verify:

- The supported agent/workflow API.
- The compatible provider package.
- Structured output support.
- How to configure model identifiers and credentials.
- Whether the chosen Gemma inference endpoint is compatible.

Do not assume the entire AI implementation can be written correctly from outdated Mastra examples.

Use Mastra for orchestration. Use a provider adapter to connect to Gemma. Avoid installing several different AI SDKs for the same purpose.

5.4 Database dependencies

Install in `packages/db`:

```
mongodb
```

Use the official MongoDB Node.js driver. Do not add Mongoose unless a specific requirement justifies it.

Configure:

- One reusable connection client.
- Collection accessors.
- Index creation.
- Repository functions.
- Graceful connection shutdown.

5.5 Game dependencies

The `packages/game` package should have **no AI provider or database dependency**.

It should contain pure TypeScript functions for:

- XP calculations.
- Level calculations.
- Bonus caps.
- Score validation.
- Ranking.
- Duplicate reward protection.

Install only the development tooling required for TypeScript and Vitest.

5.6 Shared dependencies

Install in `packages/shared`:

```
zod
```

Use this package for shared contracts, not business logic.

The frontend and backend must agree on request and response formats through shared schemas wherever practical.

5.7 Root development dependencies

Install at the monorepo root:

```
turbo
```

```
prettier
```

Configure ESLint using the current Next.js-compatible setup. Avoid maintaining incompatible, duplicated ESLint configurations.

5.8 Testing dependencies

Use:

- Vitest for unit tests.
- Supertest for Express endpoint tests.
- Playwright for browser-level end-to-end tests.

Install Playwright in the appropriate workspace and install its browser binaries during setup.

Test at minimum:

- Authentication.
- Challenge generation.
- Challenge completion.
- Evidence validation.
- XP calculation.
- Duplicate submissions.
- Space membership.
- Leaderboard updates.

5.9 Image storage dependencies

Use Cloudinary for evidence image storage.

The backend must issue secure upload parameters or provide a signed upload mechanism. Never expose the Cloudinary API secret in frontend code.

Store image metadata and secure URLs in MongoDB, not raw image binaries.

Use a development mock or local test fixture when credentials are missing.

5.10 Optional dependencies

Do not install these by default:

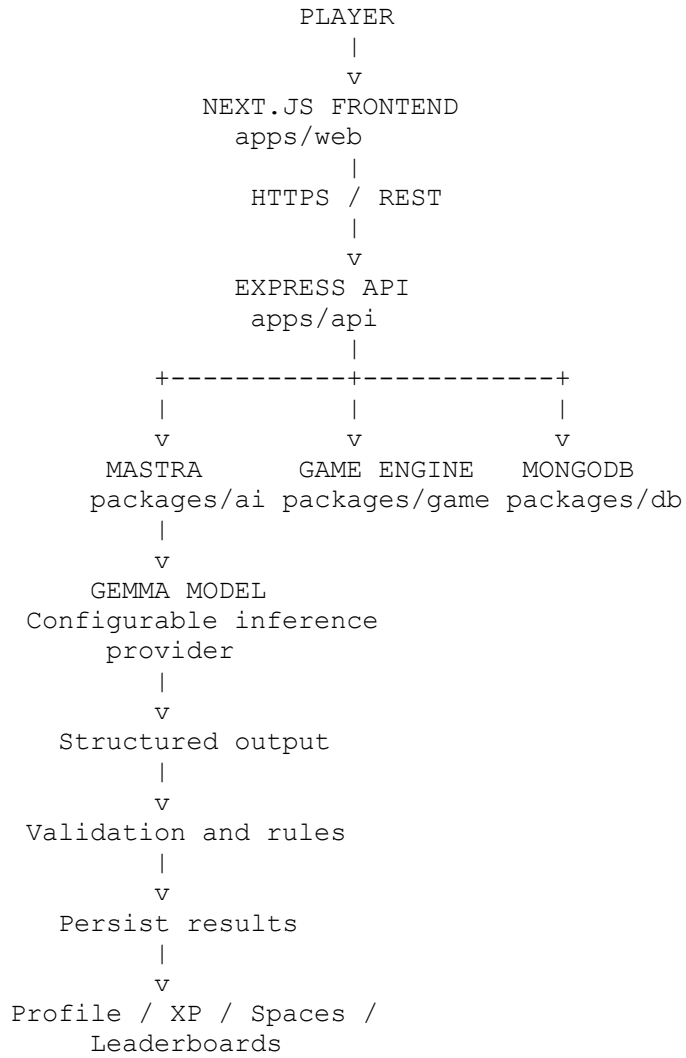
- ElevenLabs: only if a voice-based mission adds clear value and time permits.
- Temporal: only if durable scheduled workflows become necessary.
- Backboard: not required by the core architecture.
- Tinker: not required unless the team can demonstrate a useful fine-tuned model.
- Redis: not required for the initial MVP.
- Socket.IO: not required; basic polling or refetching is sufficient for Space results.
- Python/PyTorch: not required; the application is TypeScript-based and Gemma is accessed through an inference provider.

Avoid adding dependencies simply to increase the technology count.

---

6. System Architecture

6.1 High-level architecture



6.2 Challenge-generation flow

1. The player selects an experience and duration.
2. The frontend submits the request to Express.
3. Express authenticates the user and validates the input.
4. The backend retrieves relevant player history and preferences.
5. The Mastra workflow calls Gemma.
6. Gemma returns a structured challenge.
7. Zod validates the output.

8. The game engine enforces permitted challenge types, duration, verification requirements, XP bounds, and other rules.
9. The backend saves the challenge in MongoDB.
10. The frontend displays the challenge.

If model output is invalid, retry a limited number of times. If generation still fails, return a clear error or use an explicitly labelled development fallback. Never silently treat arbitrary model text as a valid challenge.

6.3 Evidence-evaluation flow

1. The player completes the real-world activity.
2. The frontend submits evidence and the challenge ID.
3. The backend checks ownership, challenge state, submission limits, and required evidence.
4. Photo evidence is uploaded securely.
5. The backend passes relevant challenge details and evidence to the AI evaluation workflow.
6. The AI returns structured evaluation scores and a proposed bounded bonus.
7. The backend validates the output.
8. The game engine calculates final XP.
9. The completion and reward are persisted atomically or with an idempotent transaction strategy.
10. The frontend displays the results and updated leaderboard.

The backend must be the authority for XP and completion status. The frontend must never determine the final score.

---

7. AI System Requirements

7.1 Challenge Generator

Create a Mastra agent or workflow dedicated to challenge generation.

Input:

- Player's chosen experience.
- Available time.
- Difficulty preference.
- Solo or Space mode.
- Relevant previous challenges.
- Optional user preferences.
- Applicable safety constraints.

Output must conform to a Zod schema.

Required challenge fields:

```
id
title
description
experienceType
durationMinutes
difficulty
verificationMethods
baseXp
maxBonusXp
instructions
safetyNotes
createdAt
```

The database should also track ownership or Space association, challenge status, and timestamps.

Supported experience types

- Calm
- Explore
- Social
- Active
- Observe
- Discover
- Surprise

Supported durations

- 10 minutes
- 20 minutes
- 30 minutes
- 60 minutes

The AI must not require users to visit dangerous, private, inaccessible, or restricted locations. Avoid instructions involving trespassing, traffic risks, abandoned buildings, unsafe encounters with strangers, or entering bodies of water.

Challenges must be feasible for the selected duration and experience.

Example challenge

Title: The Thing You Never Notice

- Experience: Observe
- Duration: 20 minutes
- Difficulty: Medium
- Verification: Photo + short text response
- Base XP: 500
- Maximum bonus: 300 XP

Instructions: Find something in your surroundings that you have probably passed before but never paid attention to. Take one photo and explain what made you notice it this time.

The AI should generate varied, specific missions rather than repeatedly producing generic instructions such as “go for a walk” or “take a picture of a tree.”

7.2 Evidence Evaluator

Create a separate AI workflow for evaluating completed challenges.

Inputs:

- Original challenge.
- Required verification methods.
- Submitted photo, where required.
- Player's written explanation, where required.
- Relevant submission metadata.

Output:

```
valid
relevanceScore
noveltyScore
interestingnessScore
explanationScore
proposedBonusXp
feedback
```

All numeric scores must be bounded and validated.

The AI should evaluate how well the evidence supports the particular challenge. It must not claim to know that an object is the rarest object in a city or make other unverifiable global claims.

A photograph alone cannot prove every claim about a player's activity. Evidence is a lightweight integrity mechanism, not perfect anti-cheat technology.

7.3 AI provider abstraction

Implement an interface that isolates the rest of the application from the selected inference provider.

Example:

```
interface AIProvider {
  generateChallenge(
    input: ChallengeGenerationInput
  ): Promise<GeneratedChallenge>;

  evaluateEvidence(
    input: EvidenceEvaluationInput
  ): Promise<EvidenceEvaluation>;
```

```
}
```

The implementation should:

- Connect to the configured Gemma inference endpoint.
- Support structured outputs or validated JSON parsing.
- Apply request timeouts.
- Handle provider errors and rate limits.
- Avoid unbounded retries.
- Log operational failures without logging secrets or sensitive evidence.
- Offer a mock implementation for development/testing.

Do not invent a Gemma API endpoint or model name. Read the current provider documentation and configure the real values through environment variables.

7.4 AI security

Treat AI-generated output and uploaded evidence as untrusted.

- Validate all generated content.
- Restrict model-selected XP values to permitted ranges.
- Do not allow the AI to execute arbitrary tools or database operations.
- Do not allow user-supplied text to override system instructions.
- Do not pass unnecessary personal data to the model.
- Do not send password hashes, JWTs, or database credentials to the model.
- Keep API credentials exclusively on the server.

---

8. Game Engine and XP

8.1 Scoring principles

Each challenge has a deterministic base XP value.

The AI may recommend a bonus, but the game engine enforces the maximum.

Example:

```
Base XP:              500
AI-proposed bonus:    245
Maximum allowed:      300
Final XP:             745
```

If the AI proposes a bonus of 900, the game engine must cap it at 300.

The final score is:

```
finalXP = baseXP + boundedBonusXP
```

Use integer XP values and validate all score inputs.

8.2 Suggested bonus model

Evaluate:

- Relevance to the challenge.
- Novelty.
- Interestingness.
- Creativity.
- Quality of the written explanation, where applicable.
- Evidence sufficiency.

The exact weights must be defined in TypeScript and documented. The model can return assessment scores, but the game engine determines the final bonus.

8.3 Challenge completion rules

- A challenge must belong to the authenticated user or an authorized Space.
- A challenge must be in an eligible state before submission.
- Required evidence must be present.
- A completion can award XP only once.
- Repeated API requests must not duplicate rewards.
- The challenge and completion status must be updated consistently.
- Invalid or incomplete submissions must not award XP.

Use an idempotency mechanism or database uniqueness constraint for reward awarding.

8.4 Levels and progression

Implement a simple, deterministic level progression system.

Requirements:

- Total XP is stored persistently.
- Level is derived from total XP using a documented formula or threshold table.
- Level calculations are deterministic.
- XP cannot be negative.
- Profile and leaderboard values update after successful completion.

Do not spend hackathon time on an elaborate economy, collectible marketplace, or complicated skill tree.

8.5 Streaks

Streaks are optional for the first working release.

If implemented:

- Track completion dates consistently.
- Define the timezone policy.
- Do not let multiple submissions on the same day artificially inflate a daily streak.
- Never make a streak the primary measure of success.

---

9. Database Design

Use MongoDB Atlas and the official Node.js driver.

The agent must implement collections, typed data contracts, repositories, and appropriate indexes.

9.1 `users`

Fields:

- `_id`
- `username`
- `email`
- `passwordHash`
- `touchGrassId`
- `totalXp`
- `level`
- `preferences`
- `createdAt`
- `updatedAt`

Requirements:

- Unique normalized email.
- Unique username, subject to the selected naming policy.
- Unique Touch Grass ID.
- Never return password hashes to clients.

9.2 `challenges`

Fields:

- `_id`
- `createdBy`
- `spaceId` (nullable)
- `experienceType`
- `title`
- `description`
- `durationMinutes`
- `difficulty`
- `verificationMethods`

- `baseXp`
- `maxBonusXp`
- `status`
- `createdAt`
- `startedAt`
- `expiresAt` (optional)

Suggested states:

```
generated
active
completed
expired
cancelled
```

Use a documented state-transition policy.

9.3 `completions`

Fields:

- `_id`
- `challengeId`
- `userId`
- `spaceId` (nullable)
- `evidenceIds`
- `valid`
- `baseXp`
- `bonusXp`
- `finalXp`
- `evaluation`
- `completedAt`

Add a uniqueness constraint appropriate to the challenge's solo or group rules so a player cannot repeatedly claim the same reward.

9.4 `evidence`

Fields:

- `_id`
- `challengeId`
- `userId`
- `type`
- `imageUrl` (optional)
- `storagePublicId` (optional)
- `textResponse` (optional)
- `submittedAt`

Do not store raw image binaries in MongoDB.

9.5 `spaces`

Fields:

- `_id`
- `name`
- `description`
- `createdBy`
- `inviteCode`
- `createdAt`

Each Space should have an owner and a membership policy.

9.6 `space_members`

Fields:

- `_id`
- `spaceId`
- `userId`
- `role`
- `joinedAt`

Roles:

- Owner
- Member

Create a unique compound index on Space and user.

9.7 Additional collections

Create a separate preferences or history collection only if required by the implemented design. Avoid overengineering the database before the core loop works.

The leaderboard should initially be calculated from persisted user XP or completion data. Do not introduce a separate denormalized leaderboard collection unless performance or consistency requirements justify it.

9.8 Database indexes

At minimum, create indexes for:

- Unique email.
- Unique username.
- Unique Touch Grass ID.
- User challenge history by user and creation date.
- Completion history by user and completion date.

- Completion lookup by challenge.
- Unique Space membership.
- Space lookup by invite code.
- Challenge status and owner.
- Unique reward/idempotency key.

The agent must document how indexes are created on startup or through a repeatable setup script.

---

10. Authentication and Authorization

Implement basic email-and-password authentication.

Required:

- Signup.
- Login.
- Logout.
- Authenticated user endpoint.
- Password hashing with bcryptjs.
- JWT creation and verification.
- Protected API routes.
- User ownership checks.
- Authorization checks for Space operations.

Use short-lived access tokens and an appropriate secure session strategy. If JWTs are used in cookies, configure HttpOnly, Secure in production, SameSite, and appropriate CSRF protection for the chosen cookie-based flow. If bearer tokens are used, do not store long-lived tokens insecurely.

The coding agent must choose one consistent session strategy and document it. Do not implement a mixture of incompatible cookie and bearer-token flows.

The frontend must:

- Handle unauthenticated states.
- Redirect protected pages appropriately.
- Show useful validation errors.
- Avoid exposing password or token data in logs.

A user must not be able to:

- Read another user's private data.
- Edit another user's profile or XP.
- Submit evidence on behalf of another user.
- Access a private Space without membership.
- Award themselves arbitrary XP through client requests.

11. REST API Specification

Use `/api/v1` as the API prefix.

11.1 Health

```
GET /api/v1/health
```

Returns service health and a basic status. Avoid exposing secrets or detailed internal infrastructure.

11.2 Authentication

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/logout
GET  /api/v1/auth/me
```

Register and login must validate all inputs and return safe user representations.

11.3 Player profile

```
GET   /api/v1/users/me
PATCH /api/v1/users/me
GET   /api/v1/users/:touchGrassId
GET   /api/v1/users/me/history
```

Public profile endpoints must expose only intended public fields.

11.4 Challenges

```
POST /api/v1/challenges/generate
GET  /api/v1/challenges/:id
POST /api/v1/challenges/:id/start
POST /api/v1/challenges/:id/complete
GET  /api/v1/challenges/me
GET  /api/v1/challenges/me/active
```

Generation accepts experience, duration, and applicable preferences.

Completion accepts evidence references and the required written responses.

11.5 Evidence

```
POST /api/v1/evidence/upload-signature
POST /api/v1/evidence
GET  /api/v1/evidence/:id
```

The upload-signature endpoint must require authentication and issue only narrowly scoped upload credentials.

The backend must verify that the stored evidence belongs to the authenticated user and is associated with the correct challenge before evaluation.

11.6 Leaderboard

```
GET /api/v1/leaderboard
GET /api/v1/spaces/:id/leaderboard
```

Support pagination or a sensible result limit. Do not return the entire user collection.

11.7 Spaces

```
POST /api/v1/spaces
GET  /api/v1/spaces
GET  /api/v1/spaces/:id
POST /api/v1/spaces/join
POST /api/v1/spaces/:id/leave
GET  /api/v1/spaces/:id/members
POST /api/v1/spaces/:id/challenges
GET  /api/v1/spaces/:id/challenges
```

Use a generated invitation code or Touch Grass ID-based invite flow. Never allow a user to join a private Space simply by guessing a database ID.

Every endpoint must:

- Validate input.
- Authenticate the user where required.
- Check authorization.
- Return consistent response formats.
- Handle errors safely.

---

12. Solo and Multiplayer Experience

12.1 Solo mode

A player:

1. Opens the home page.
2. Selects an experience.
3. Selects a duration.
4. Receives an AI-generated challenge.
5. Starts the challenge.
6. Goes outside.

7. Returns and submits evidence.
8. Receives XP and feedback.
9. Sees their updated profile and ranking.

Solo mode must work without creating or joining a Space.

12.2 Spaces

A Space is a shared challenge room for friends, not a full social network.

Players can:

- Create a Space.
- Choose a name.
- Invite friends using a shareable code or Touch Grass ID.
- View members.
- Start a group challenge.
- Submit their results independently.
- View a ranked results screen.

12.3 Group challenge lifecycle

1. An authorized Space member creates a group challenge.
2. The challenge is associated with the Space.
3. Eligible members can participate.
4. Each participant completes the challenge and submits evidence.
5. Each submission is evaluated using the same scoring rubric.
6. The backend calculates results.
7. The Space leaderboard displays rankings.

For the MVP, a group challenge can be asynchronous. Friends do not need to be online simultaneously or in the same physical location.

Do not require real-time multiplayer infrastructure for the first release.

---

13. Frontend Screens and Requirements

13.1 Landing page

Explain the concept clearly:

- AI-generated outdoor challenges.
- Solo and friend play.
- XP and progression.
- The phone-away philosophy.

Provide clear signup and login actions.

13.2 Home page

Display:

- Player name and level.
- Total XP.
- Current streak, if implemented.
- A prominent challenge-generation action.
- Experience choices.
- Active challenge, if one exists.
- Space activity.
- Leaderboard preview.

13.3 Experience selection

Options:

- Calm.
- Explore.
- Social.
- Active.
- Observe.
- Discover.
- Surprise Me.

Let the player select a duration of 10, 20, 30, or 60 minutes.

Keep the interaction quick. Do not force players through a lengthy questionnaire before every challenge.

13.4 Challenge page

Display:

- Challenge title.
- Description.
- Duration.
- Difficulty.
- Required evidence.
- Base XP.
- Maximum bonus XP.
- Safety notes where relevant.
- Start button.

13.5 Active challenge page

The active experience should be deliberately minimal.

Display:

- The essential instructions.
- A clear reminder to put the phone away.
- A way to end or abandon the challenge.

Do not build continuous GPS tracking or require the player to interact with the screen throughout the activity.

13.6 Completion page

Allow the player to:

- Upload a photo when required.
- Enter a short response when required.
- Review the evidence.
- Submit their completion.

Show progress and useful validation errors.

13.7 Results page

Display:

- Challenge completed.
- Base XP.
- AI bonus XP.
- Final XP.
- Brief feedback explaining the evaluation.
- Updated level and XP.
- Updated leaderboard position where available.
- A next-challenge action.

Do not expose private internal prompts or chain-of-thought. The explanation should be a short user-facing rationale, not the model's hidden reasoning.

13.8 Profile page

Display:

- Username.
- Touch Grass ID.
- Level.
- Total XP.
- Completed challenges.
- Challenge history.

- Favorite experience types.
- Discovery collection, if implemented.

13.9 Leaderboard page

Display:

- Ranked players.
- Total XP.
- Level.
- The current player's position.
- Space-specific rankings when applicable.

13.10 Spaces pages

Support:

- Space creation.
- Joining a Space.
- Members.
- Active and completed group challenges.
- Submission status.
- Group results.

Use responsive layouts and loading, empty, and error states on all pages.

---

14. Evidence and Photo Handling

Photo evidence is an important mechanic, but not every challenge should require a photograph.

Supported evidence types:

- Photo.
- Short text response or memory.
- Observation.
- Activity data, if later implemented.
- Location confirmation, if later implemented.
- Honor-system completion for subjective challenges.

Use only the verification methods appropriate to each challenge.

14.1 Photo upload requirements

- Use Cloudinary for storage.

- Validate MIME type and file size.
- Restrict uploads to supported image formats.
- Generate signed upload parameters on the backend.
- Store image metadata and URL in MongoDB.
- Do not store API secrets in the browser.
- Reject unsupported file types.
- Handle failed uploads gracefully.

The exact maximum image size must be configured and documented.

14.2 Privacy

- Request camera or location access only when needed.
- Do not require continuous location tracking.
- Do not expose private evidence URLs unnecessarily.
- Avoid capturing identifiable bystanders where possible.
- Provide clear feedback on how evidence is used.
- Do not make uploaded images publicly browsable by default.

---

15. Security and Reliability

Implement the following baseline controls:

1. Validate all request bodies, path parameters, and query parameters.
2. Use Helmet.
3. Restrict CORS to configured origins.
4. Rate-limit authentication, challenge generation, upload-signature, and evaluation endpoints.
5. Protect private endpoints with authentication.
6. Enforce ownership and Space membership.
7. Store secrets in environment variables.
8. Never commit `.env` files.
9. Never trust XP, user IDs, role values, or challenge status supplied by the frontend.
10. Prevent duplicate XP awards.
11. Limit AI retries and handle model timeouts.
12. Use safe, consistent error responses.
13. Avoid returning stack traces to production clients.
14. Ensure application shutdown closes the database connection.
15. Log operational errors without logging passwords, tokens, or unnecessary evidence content.

The backend must enforce all critical game rules independently of the frontend.

---

16. Environment Variables

Create a root `.env.example` and document which application consumes each variable.

Use placeholders only. Never write real credentials into the repository.

Expected variables include:

```
# Runtime
NODE_ENV=development

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1

# Backend
API_PORT=4000
WEB_ORIGIN=http://localhost:3000

# MongoDB
MONGODB_URI=
MONGODB_DATABASE=touch_grass

# Authentication
JWT_SECRET=
JWT_EXPIRES_IN=15m

# Gemma inference provider
AI_PROVIDER=
AI_BASE_URL=
AI_API_KEY=
AI_MODEL=
AI_REQUEST_TIMEOUT_MS=30000

# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# Optional
LOG_LEVEL=info
```

The agent must validate required environment variables at startup and report missing configuration clearly.

Rules:

- `NEXT_PUBLIC_*` variables must never contain secrets.
- Only require Cloudinary credentials when the real upload feature is enabled.
- Only require AI credentials when the real provider is enabled.
- Use separate environment configuration for local development and production.
- Do not commit actual `.env` files.

If a selected Gemma provider needs a different authentication mechanism or endpoint format, adjust the provider configuration accordingly.

17. Local Development and Setup

The coding agent must create all files and execute all necessary commands, not merely provide instructions.

17.1 Prerequisites

Verify:

- Node.js LTS compatible with the selected Next.js release.
- pnpm available.
- Git available.
- Network access for package installation.

Report any missing prerequisites.

17.2 Required setup steps

1. Create the root project directory.
2. Initialize the pnpm monorepo.
3. Create the Next.js frontend.
4. Create the Express API with TypeScript.
5. Create the `ai`, `db`, `game`, and `shared` packages.
6. Configure workspace package names and dependencies.
7. Configure TypeScript.
8. Configure Turborepo.
9. Install all required dependencies.
10. Configure linting and formatting.
11. Add environment validation.
12. Implement MongoDB connection and indexes.
13. Implement the Express health endpoint.
14. Implement the AI provider abstraction and Mastra workflow.
15. Implement game scoring and validation.
16. Implement authentication.
17. Implement challenge generation and completion.
18. Implement evidence storage and evaluation.
19. Implement Spaces and leaderboards.
20. Implement the frontend pages.
21. Add tests.
22. Verify local development and production builds.
23. Prepare deployment configuration and documentation.

17.3 Development commands

The finished repository must support commands equivalent to:

```
pnpm install
pnpm dev
pnpm build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:e2e
```

`pnpm dev` should start the frontend and backend together.

Expected local addresses:

```
Frontend: http://localhost:3000
Backend:  http://localhost:4000
Health:   http://localhost:4000/api/v1/health
```

The application must not require production credentials merely to render the initial frontend or execute unit tests.

---

18. Deployment

18.1 Hosting

Use Render for deployment.

Deploy:

- The Next.js web application.
- The Express API as a separate service.

Use MongoDB Atlas for the database and Cloudinary for evidence storage.

18.2 Production requirements

- Configure environment variables in Render.
- Restrict CORS to the deployed frontend domain.
- Use HTTPS.
- Set production cookie/security settings if cookie-based authentication is used.
- Ensure the API listens on the configured port.
- Configure the frontend's API URL.
- Ensure workspace packages build correctly in the deployment environment.
- Add a health-check endpoint.
- Verify database connectivity from the deployed API.
- Verify Gemma inference from the deployed API.
- Test photo upload and evaluation in production.
- Do not rely on local disk for persistent user uploads.

The coding agent must provide exact Render configuration and build/start commands based on the completed repository. Do not assume a monorepo deployment will work with default settings without testing it.

---

19. Testing and Acceptance Criteria

The project is not complete merely because it compiles or the frontend looks good.

19.1 Build and configuration

- [ ] `pnpm install` completes successfully.
- [ ] Workspace dependencies resolve correctly.
- [ ] `pnpm dev` starts both frontend and API.
- [ ] `pnpm build` succeeds.
- [ ] `pnpm typecheck` succeeds.
- [ ] `pnpm lint` succeeds.
- [ ] `pnpm test` succeeds.
- [ ] No committed secrets.
- [ ] `.env.example` is documented.

19.2 Authentication

- [ ] A user can register.
- [ ] A user can log in.
- [ ] Invalid credentials are rejected.
- [ ] Passwords are hashed.
- [ ] Protected endpoints reject unauthenticated requests.
- [ ] Users cannot edit other users' data.

19.3 AI challenge generation

- [ ] A player can select an experience.
- [ ] A player can select a duration.
- [ ] A real configured Gemma provider can generate a challenge.
- [ ] Output is validated against a schema.
- [ ] Invalid AI output is rejected or retried safely.

- [ ] Challenges are persisted.
- [ ] The application handles AI outages gracefully.
- [ ] Previous challenges can be used to reduce repetition.

19.4 Evidence and scoring

- [ ] A player can submit required evidence.
- [ ] Photo uploads are securely handled.
- [ ] Evidence is evaluated using the configured AI provider.
- [ ] AI scores are validated.
- [ ] Bonus XP is bounded.
- [ ] The game engine calculates the final score.
- [ ] Repeated completion requests do not duplicate XP.
- [ ] Invalid submissions do not award XP.
- [ ] Completion history persists.

19.5 Spaces and leaderboards

- [ ] A user can create a Space.
- [ ] Another user can join through the invitation mechanism.
- [ ] Unauthorized users cannot access private Space data.
- [ ] Members can participate in group challenges.
- [ ] Results are persisted.
- [ ] Rankings reflect awarded XP.
- [ ] Solo mode works independently of Spaces.

19.6 Frontend

- [ ] All major screens are implemented.
- [ ] Forms have validation and useful errors.
- [ ] Loading and empty states are implemented.
- [ ] API errors are handled.
- [ ] The interface works on mobile and desktop.
- [ ] Active challenges encourage minimal phone interaction.
- [ ] No screen depends on mock data when real data is available.

19.7 End-to-end scenario

The following scenario must work from beginning to end:

1. Register two test users.
2. Create a Space with one user.
3. Join the Space with the second user.
4. Generate a group challenge.
5. Submit evidence for a challenge.
6. Evaluate evidence using Gemma or a clearly identified test provider.
7. Award XP once.
8. Verify the updated profile.
9. Verify the Space leaderboard.
10. Reload the application and confirm that the data persists.

If the real model provider is unavailable, the agent must distinguish passing mock-based tests from verified real-model integration.

---

20. MVP Scope and Priorities

Priority 1 — Required for a working product

- Frontend and backend.
- Authentication.
- Persistent player profiles.
- Experience selection.
- AI-generated challenges.
- Structured challenge validation.
- Challenge lifecycle.
- Photo and text evidence.
- AI evidence evaluation.
- Deterministic XP scoring.
- Basic levels.
- Leaderboard.
- Spaces and invitation flow.
- Group challenges.
- Database persistence.
- Deployment.
- Tests and setup documentation.

Priority 2 — Add if time permits

- Streaks.
- Discovery collection.
- Activity tracking.

- Optional location confirmation.
- Richer progression and achievements.
- Voice instructions using ElevenLabs.
- Scheduled weekly challenges.

Explicitly out of scope for the MVP

- Complex social feeds.
- Direct messaging.
- Elaborate guild systems.
- Full GPS route navigation.
- Continuous background tracking.
- Complex fitness analytics.
- Payments or virtual currency purchases.
- Real-time multiplayer infrastructure.
- AI-generated voice for every challenge.
- Temporal workflows without a demonstrated need.

The agent should prioritize a working end-to-end game loop over optional features.

---

21. Product Differentiation

The core differentiator is not simply that an AI generates outdoor tasks.

Touch Grass combines:

1. **Personalized challenges:** The player chooses the experience they want.
2. **Real-world execution:** The activity takes place outside the app.
3. **Flexible evidence:** Verification depends on the challenge.
4. **Bounded AI evaluation:** The AI assesses evidence while deterministic rules control XP.
5. **Persistent progression:** Players accumulate XP and levels.
6. **Solo and social play:** Players can participate alone or compete through Spaces.
7. **Phone-away design:** The application is designed to be used briefly before and after the activity.

The product should feel like a persistent game that uses the real world as its playing field, not an AI walking companion.

---

22. Instructions for the Coding Agent

Treat this document as the source of truth for the initial implementation.

Execution rules

1. Inspect the existing repository before making changes. Preserve existing work where possible.
2. Use TypeScript and pnpm throughout the project.
3. Use Next.js for the frontend and Node.js + Express for the backend.
4. Use a pnpm monorepo with `apps/web`, `apps/api`, and the four shared packages.
5. Install compatible dependencies and verify package APIs against current documentation.
6. Create real source files and working scripts, not just empty folders.
7. Configure workspace dependencies correctly.
8. Keep AI orchestration, database access, game logic, and shared types separated.
9. Use Gemma through a configurable, documented inference provider.
10. Never invent provider credentials, model endpoints, or successful integration results.
11. Implement real database persistence rather than relying on hard-coded sample data.
12. Implement real authentication and authorization.
13. Keep XP calculations deterministic and prevent duplicate rewards.
14. Use environment variables for secrets.
15. Add tests for critical business logic and API flows.
16. Run installation, type-checking, linting, tests, and builds.
17. Fix errors introduced by the implementation before declaring completion.
18. Document every external account, credential, or manual configuration step that remains.
19. If a required external service cannot be configured, implement a clear adapter boundary and explain what is blocked.
20. Do not add optional technologies unless they solve a concrete MVP requirement.

Required final report from the agent

When implementation is complete, report:

- The repository structure created.
- Dependencies installed.
- Scripts available.
- Local development commands.
- Environment variables that must be configured.
- External accounts/services that still require setup.
- Whether MongoDB connectivity was tested.
- Whether real Gemma inference was tested.
- Whether Cloudinary uploads were tested.
- Tests and builds that passed or failed.
- Any remaining issues.
- Exact deployment instructions.

Completion means the application runs, its core flows work, and the remaining external setup is explicitly documented. Do not claim that a service works if it has only been mocked.

---

23. Final Architecture Summary

```
TOUCH GRASS MONOREPO
│
├── apps/
│   ├── web
│   │   └── Next.js + React + TypeScript + Tailwind
│   │
│   └── api
│       └── Node.js + Express + TypeScript
│
├── packages/
│   ├── ai
│   │   └── Mastra + Gemma provider + AI workflows
│   │
│   ├── db
│   │   └── MongoDB Atlas + repositories + indexes
│   │
│   ├── game
│   │   └── XP + levels + validation + scoring
│   │
│   └── shared
│       └── TypeScript contracts + Zod schemas
│
├── pnpm-workspace.yaml
├── turbo.json
├── tsconfig.base.json
├── .env.example
├── package.json
└── README.md
```

Final technology decisions:

- Frontend: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui.
- Backend: Node.js, Express.js, TypeScript.
- AI: Mastra orchestrating Gemma through a configurable inference provider.
- Database: MongoDB Atlas.
- Game logic: deterministic TypeScript.
- Evidence storage: Cloudinary.
- Package manager: pnpm.
- Monorepo tooling: Turborepo.
- Testing: Vitest, Supertest, Playwright.
- Deployment: Render.

The agent should build this as one coherent application with a single working gameplay loop, rather than as a collection of disconnected demos.